

Lecture 6

Perceptron Pt.2 + Backpropagation

Elena Raponi | Neural Computing course



Universiteit
Leiden
The Netherlands

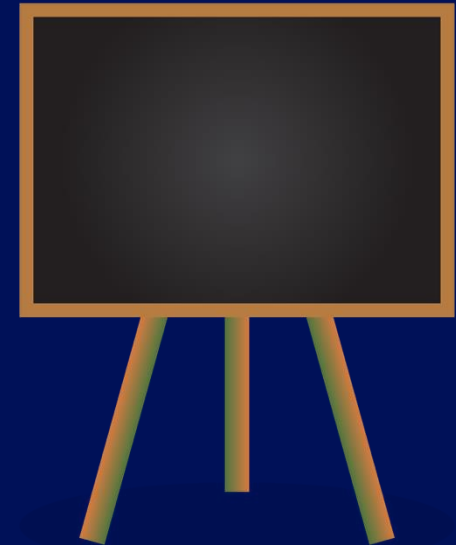
Discover the world at Leiden University

Index

1. Proof of Convergence of the Learning Rule
2. Multi-layer Perceptron
3. Gradient Field
4. Recap Derivatives
5. Gradient Descent
6. Back-propagation Algorithm
7. Gradient Descent algorithms (Momentum)



Proof of Convergence of the Learning Rule on the blackboard



How about non-linearly separable data?

The eXclusive OR problem (XOR)

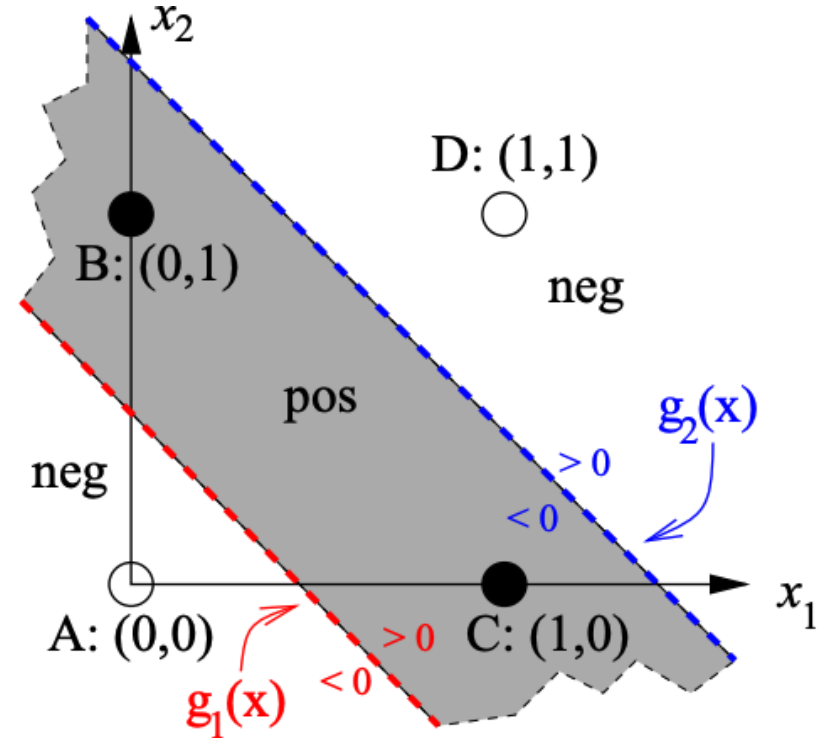
- Cannot be solved with one decision boundary/function
- What about two?

$$g_1(\vec{x}) = 1 \cdot x_1 + 1 \cdot x_2 - 1/2$$

$$g_2(\vec{x}) = 1 \cdot x_1 + 1 \cdot x_2 - 3/2$$

Positive = $g_1(\vec{x}) > 0$ AND $g_2(\vec{x}) < 0$

Negative = $g_1(\vec{x})g_2(\vec{x}) > 0$



How about non-linearly separable data?

The eXclusive OR problem (XOR)

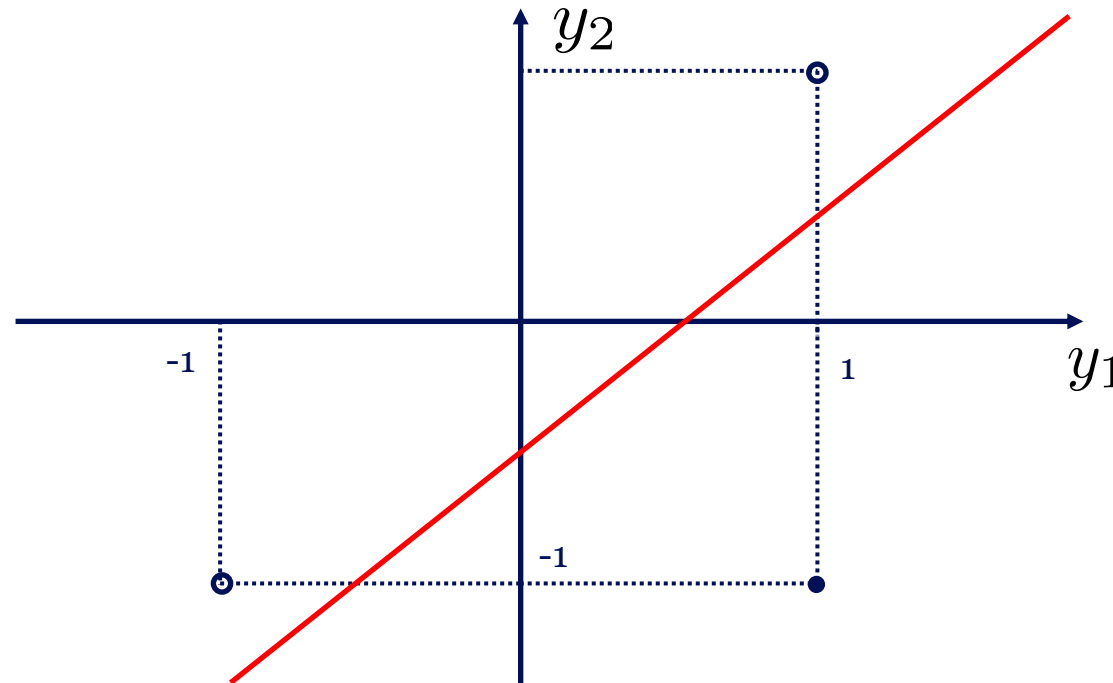
- What if we pass g_1, g_2 into a threshold function?

$$g_1(\vec{x}) = 1 \cdot x_1 + 1 \cdot x_2 - 1/2$$

$$g_2(\vec{x}) = 1 \cdot x_1 + 1 \cdot x_2 - 3/2$$

$$y_1 = \text{sgn}(g_1(\vec{x})), y_2 = \text{sgn}(g_2(\vec{x}))$$

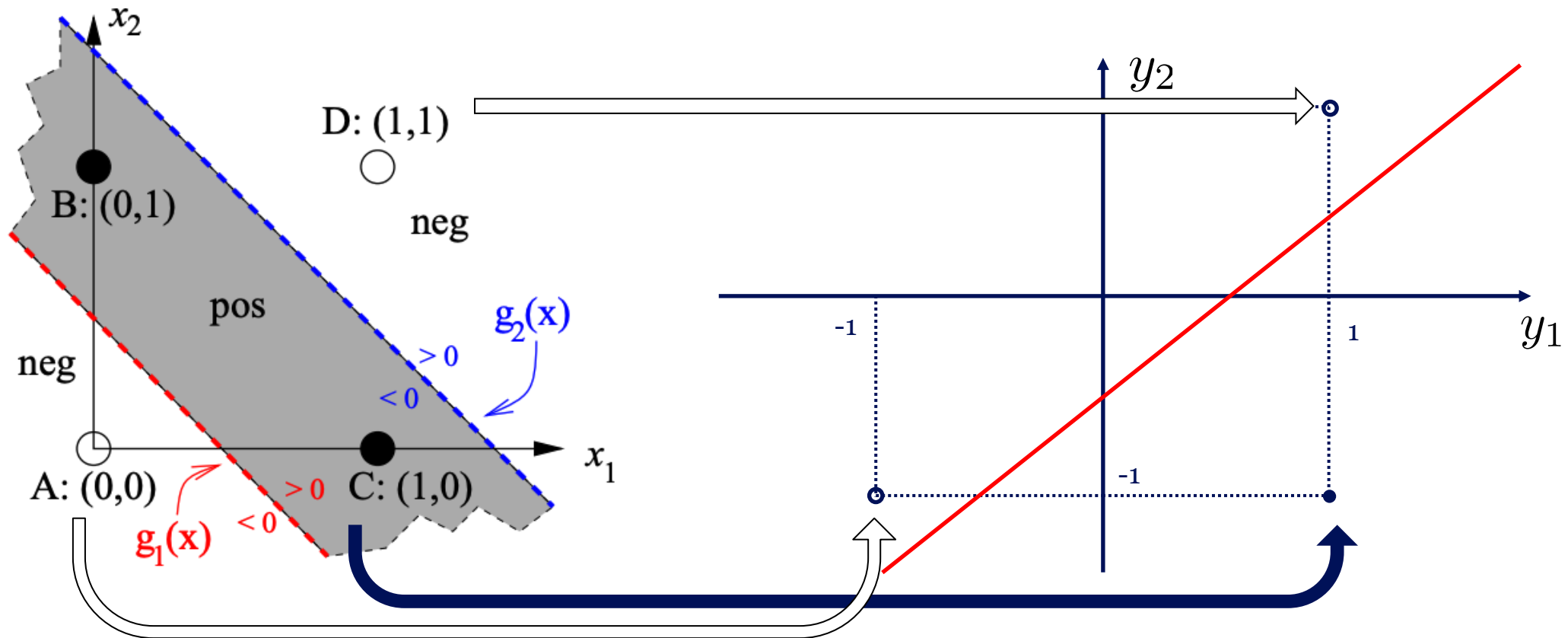
- Linearly separable!



How about non-linearly separable data?

The eXclusive OR problem (XOR)

- We map all vectors $\vec{x}$ to $\vec{y}$ such that the classes are linearly separable in the new space

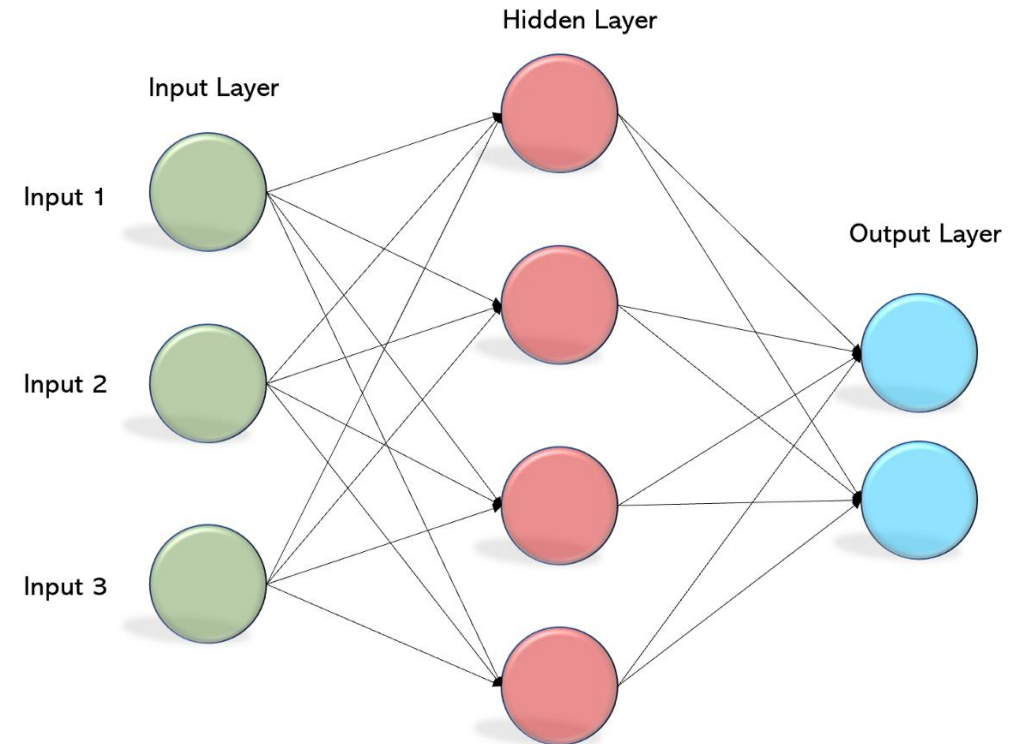


Multi-Layer Perceptron

How to implement this?

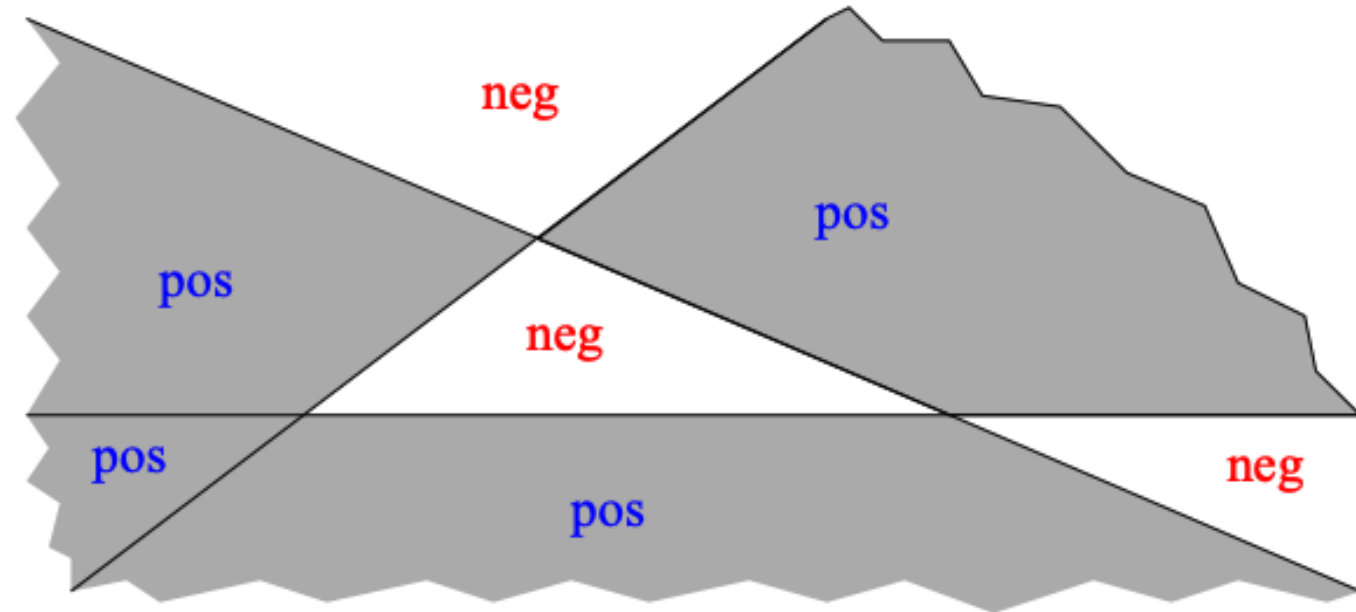
Multi-layer Perceptron

- The hidden layer functions to ***extract features***
- The output layer (last layer) implements the decision function



Multi-Layer Perceptron

By adding up to 2 hidden layers of perceptrons, can represent any union of intersection of half-spaces.



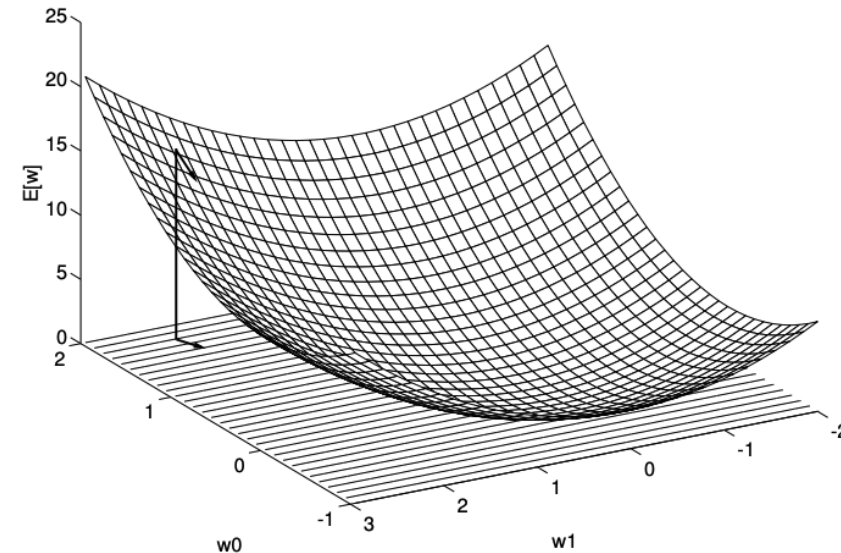
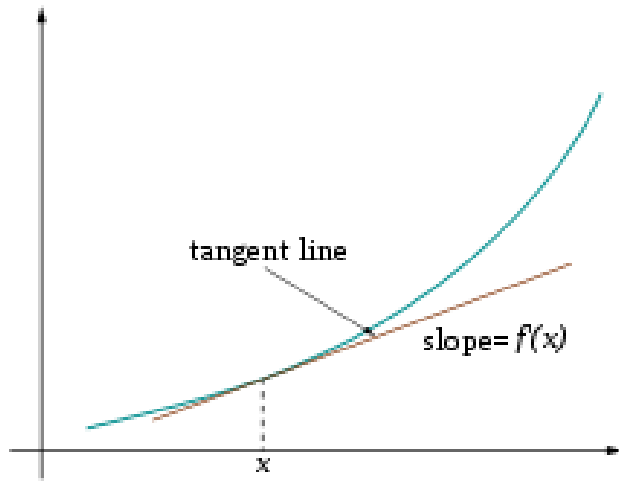
Multi-Layer Perceptron - how to train?

Perceptron revisited

Single perceptron: $O^p = \text{sgn}(\vec{w}^T \vec{x}^p + b), \quad p = 1, \dots, P$

Define a loss function: $L(\vec{w}, \vec{x}^p) = \frac{1}{2} (y_p - \vec{w}^T \vec{x}^p)^2$ (we here consider the case of linear units: $g(h) = h$)

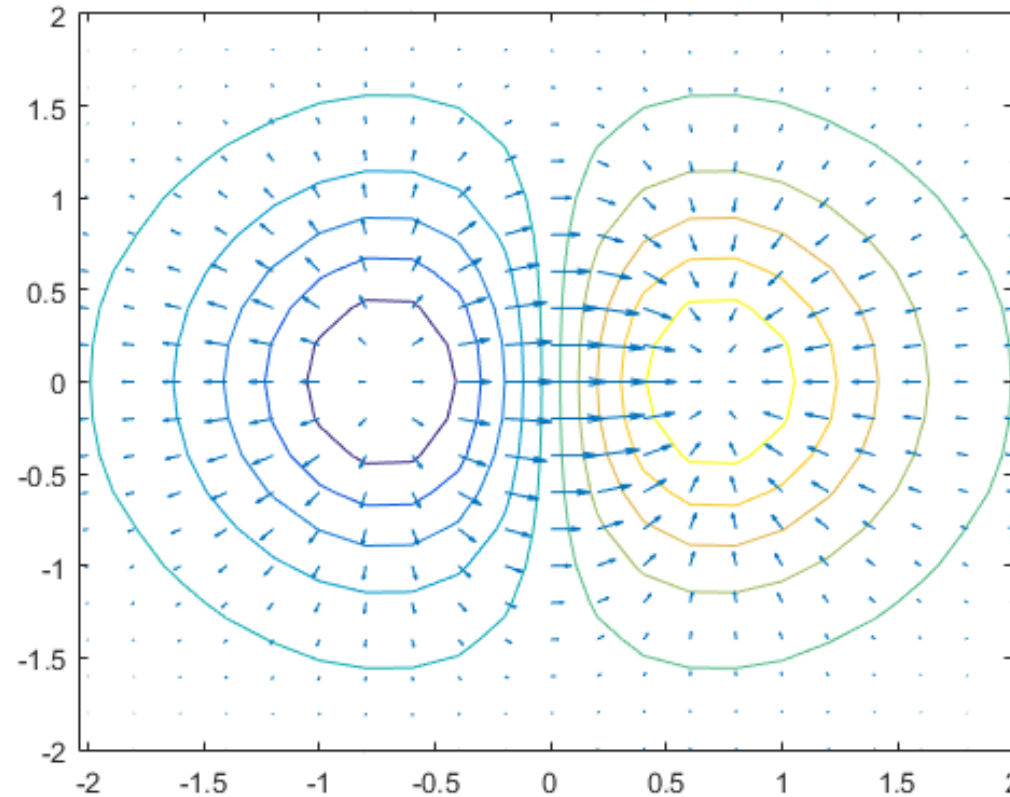
How does the loss change w.r.t. the weights? – gradients! $\rightarrow \frac{\partial L}{\partial w_k}$



Gradient field - Example

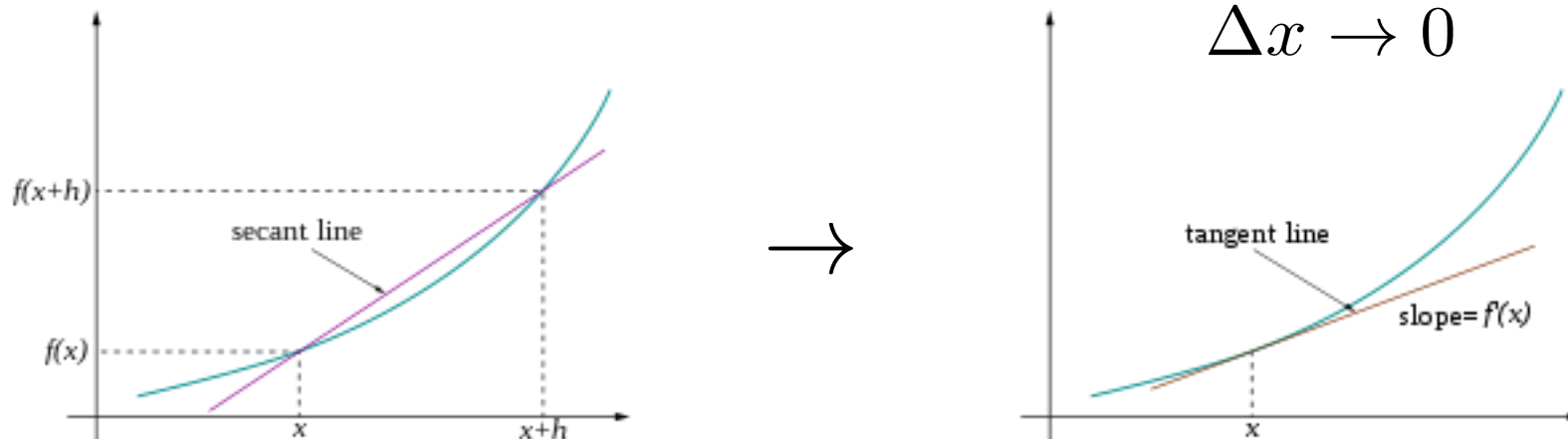
$$\nabla L = \frac{\partial L}{\partial \vec{w}} = \left(\frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_d} \right)^\top$$

$$f(x, y) = xe^{-x^2-y^2}$$



Recap - Derivatives

- Univariate function: $f: \mathbb{R} \rightarrow \mathbb{R}, x \mapsto f(x)$
 - e.g., $f(x) = e^x$
 - Derivative is the slope at a point on the graph of the function
- Defined as a limit: $f' = \frac{df}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$



Recap - Derivatives

Multivariate function $f: \mathbb{R}^n \rightarrow \mathbb{R}, (x_1, x_2, \dots, x_n) \mapsto y$

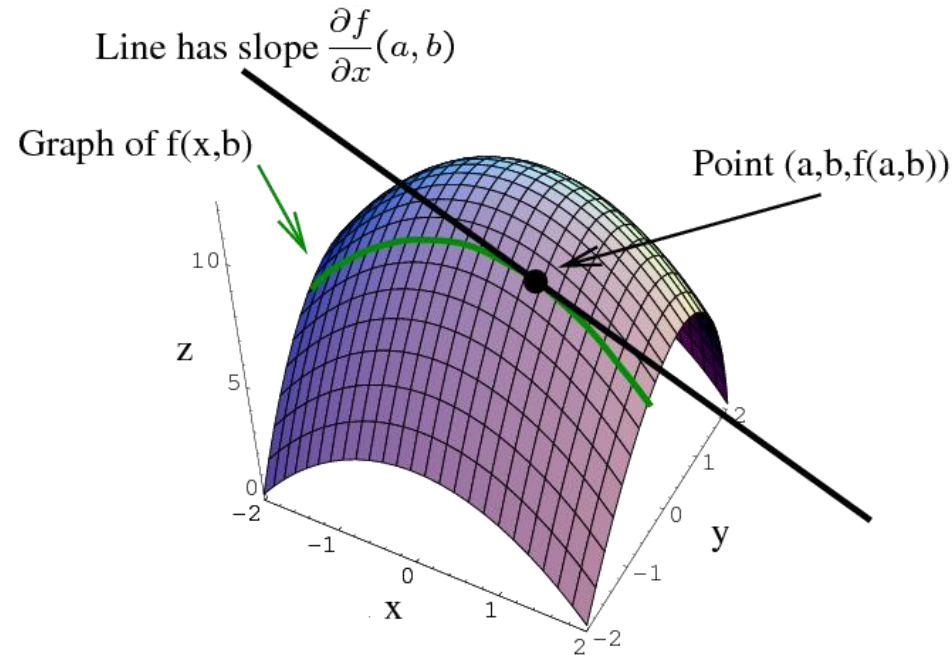
- Perceptron's output: $O = \text{sgn} \left(\sum_{i=1}^n w_i x_i \right)$

- **Partial** derivative w.r.t. x_i is: $\frac{\partial f}{\partial x_i} = \lim_{\varepsilon \rightarrow 0} \frac{f(\vec{x} + \varepsilon \vec{e}_i) - f(\vec{x})}{\varepsilon}$

- The slope (rate of change) of the function along the i -th axis/basis of $\mathbb{R}^n$

Recap - Derivatives

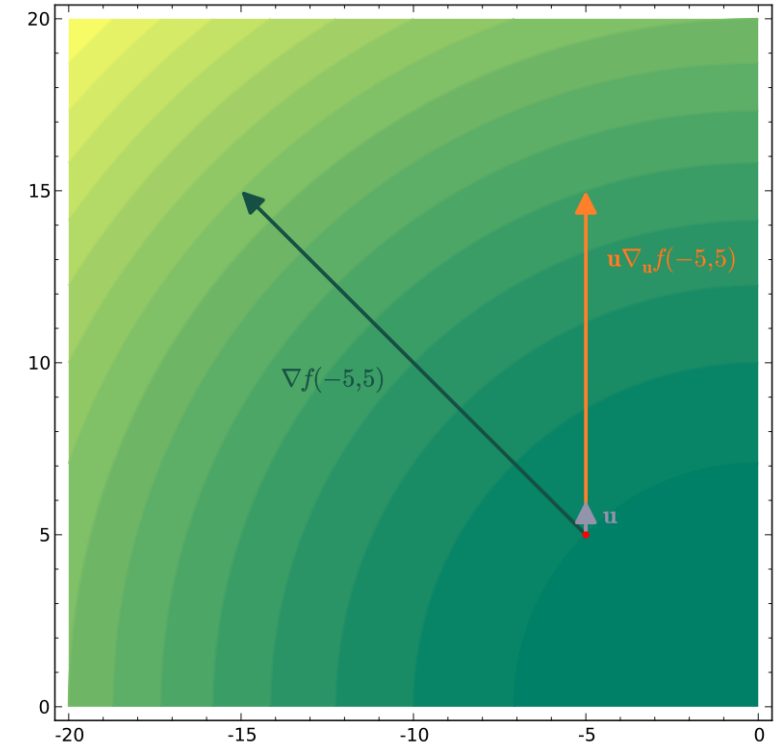
- **Partial** derivative w.r.t. x_i is: $\frac{\partial f}{\partial x_i} = \lim_{\varepsilon \rightarrow 0} \frac{f(\vec{x} + \varepsilon \vec{e}_i) - f(\vec{x})}{\varepsilon}$
- The slope (rate of change) of the function along the i -th axis/basis of $\mathbb{R}^n$
- How much will the function value change if we only perturb x_i a bit?



Recap – Gradient

Gradient is simply a vector collecting all partial derivatives

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^\top$$



- The **steepest** ascending direction $\frac{\nabla f(\vec{x})}{\|\nabla f(\vec{x})\|} = \arg \max_{\|\vec{d}\|=1} f(\vec{x} + \vec{d}) - f(\vec{x})$

The basic rules and the chain rule

For univariate functions:

$$(f(x) + g(x))' = f(x)' + g(x)'$$
$$(fg)' = f'g + fg'$$
$$(g(f(x)))' = g(f(x))' f'(x)$$

For multivariate functions:

$$O = g(f(\vec{x}))$$
$$\frac{\partial O}{\partial x_i} = g(f(\vec{x}))' \frac{\partial f}{\partial x_i}$$
$$O = g(\vec{f}(\vec{x}))$$
$$\frac{\partial O}{\partial x_i} = \sum_{k=1}^d \frac{\partial O}{\partial f_k} \frac{\partial f_k}{\partial x_i}$$

Multi-Layer Perceptron - how to train?

Perceptron revisited

Gradient is the steepest direction along which the loss increases

$$\begin{aligned}\frac{\partial L}{\partial w_k} &= \frac{\partial}{\partial w_k} \frac{\left(y_p - \sum_{j=1}^d w_j x_j^p\right)^2}{2} \\ &= -\left(y_p - \sum_{j=1}^d w_j x_j^p\right) x_k^p \\ &= -(y_p - o_p) x_k^p\end{aligned}$$

Does it look familiar? 😊

Multi-Layer Perceptron - how to train?

Perceptron revisited

Using the gradient to update the weights:

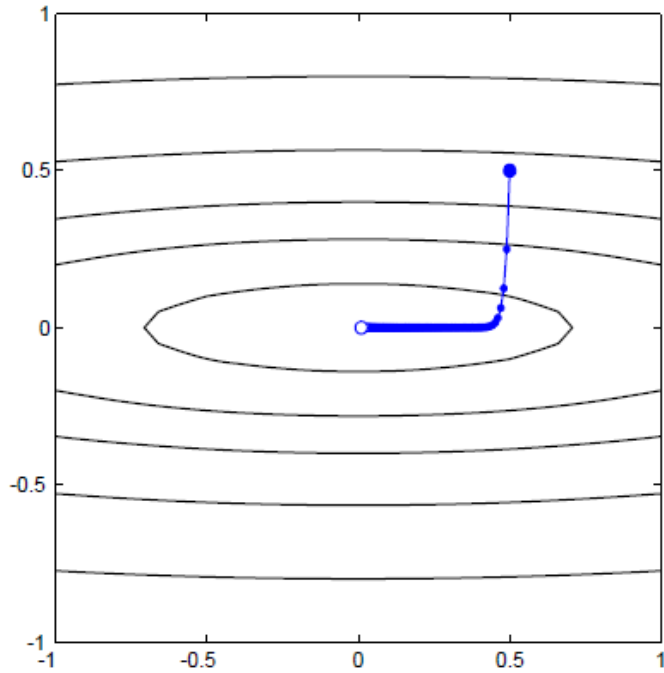
$$w_k \leftarrow w_k - \eta \frac{\partial L}{\partial w_k}$$
$$\Delta w_k = -\eta \frac{\partial L}{\partial w_k} = \eta (y_p - o_p) x_k^p$$

- We want to minimize the loss, hence subtracting the gradient from the weights – **Gradient Descent**
- The step-size η is necessary (next slides)
- We re-discover Hebb's rule!

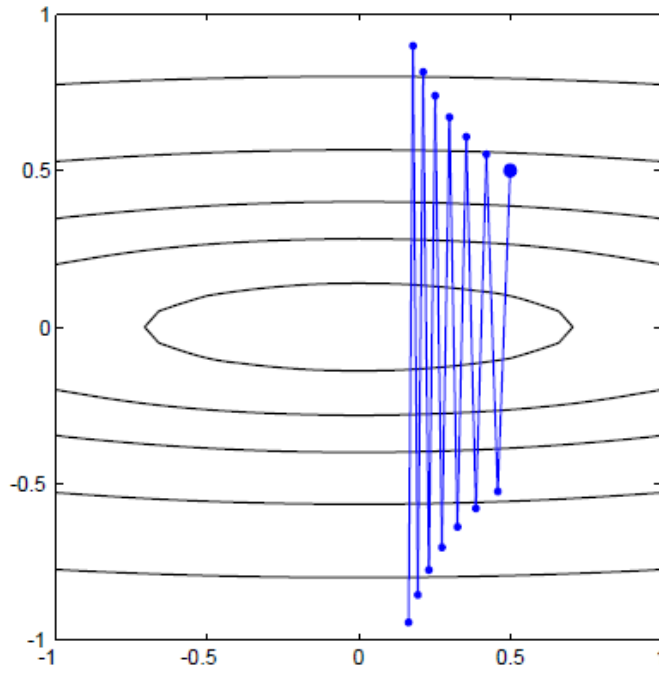
Effect of the step-size: example

$$f(x, y) = xe^{-x^2-y^2}$$

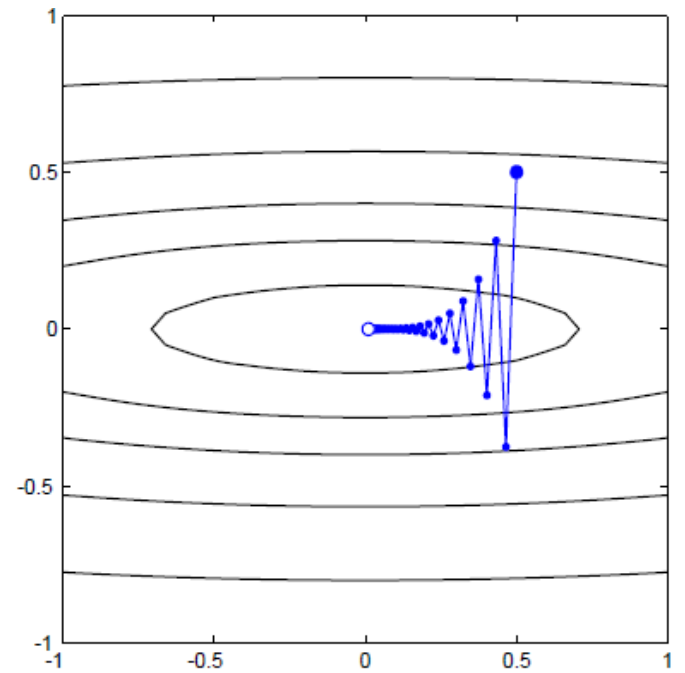
Trajectory of gradient descent with...



$\eta = 0.01$

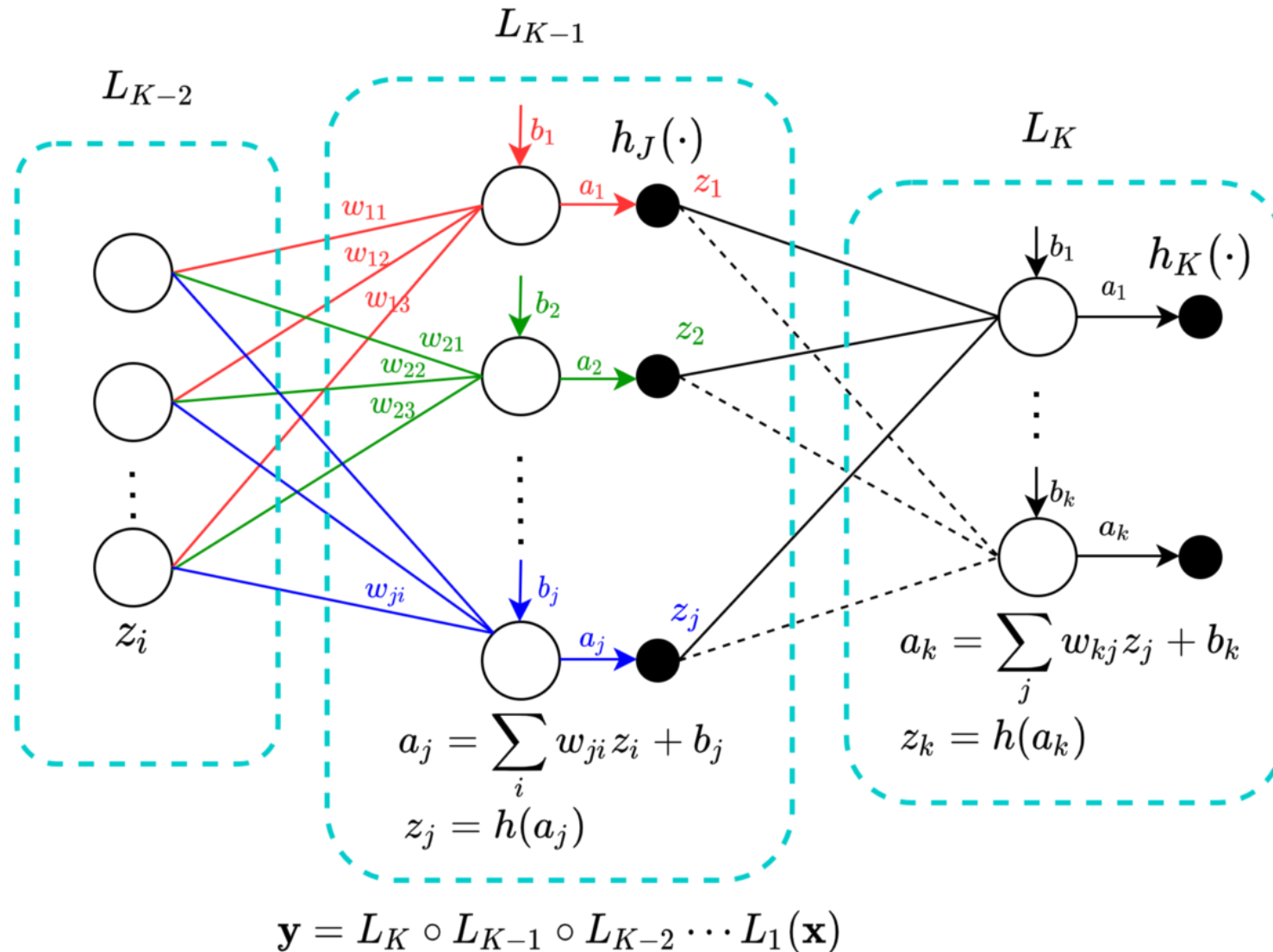


$\eta = 0.041$



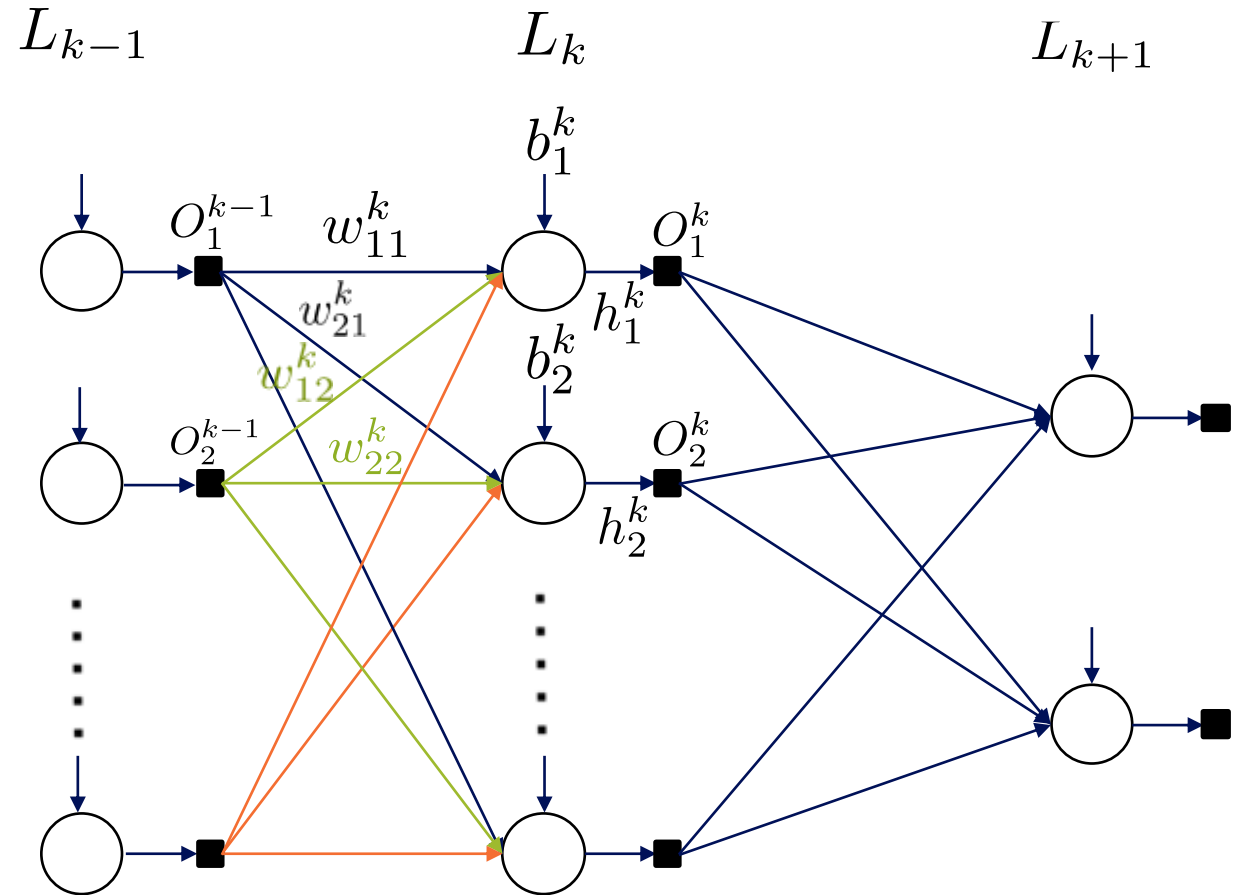
$\eta = 0.035$

Okay... how to apply GD to multiple layers?

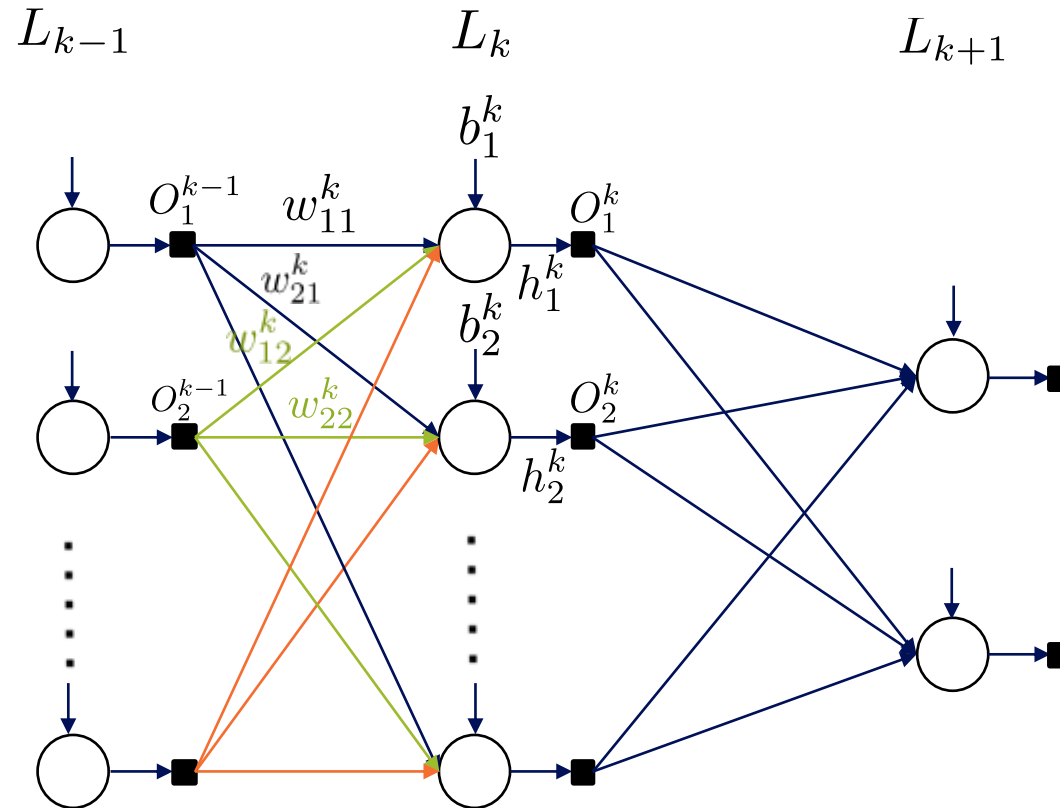


Notation leveraged

- $w_{ij}^{(k)}$: weight for node i in layer k for incoming input j
- $b_i^{(k)}$: bias for node i in layer k
- $h_i^{(k)}$: sum of inputs for node i in layer k
- $O_i^{(k)}$: output for node i in layer k
- n_k : number of nodes in layer k



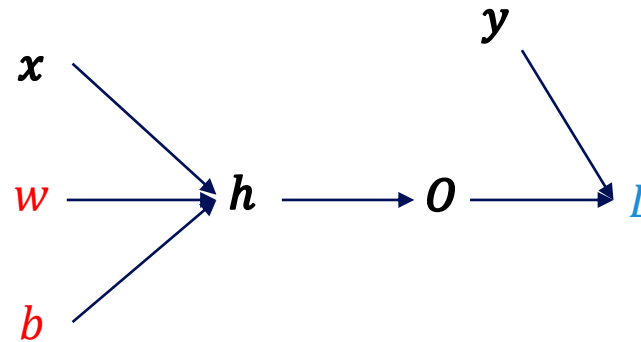
Notation leveraged



$$O_i^{(k)} = g(h_i^{(k)}) = g \left(\sum_{j=1}^{n_{k-1}} w_{ij} O_j^{(k-1)} + b_i^{(k)} \right)$$

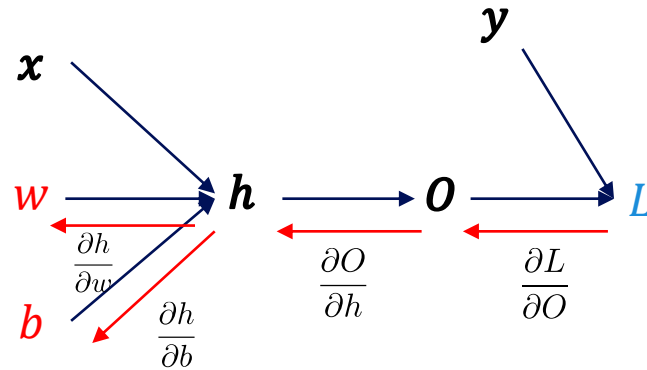
Step 1 – Computation Graph

- Consider a single node with only one input
- We represent it as a **computation graph**
 - Each node – a quantity in a multi-layer perceptron (MLP)
 - Arrow – input/output relation
 - Red nodes – variables of MLPs



Step 2 - Univariate Chain Rule

Consider a single node with only one input



Forward pass:

$$h = wx + b$$

$$O = g(h)$$

$$L = \frac{1}{2}(y - O)^2$$

Backward pass:

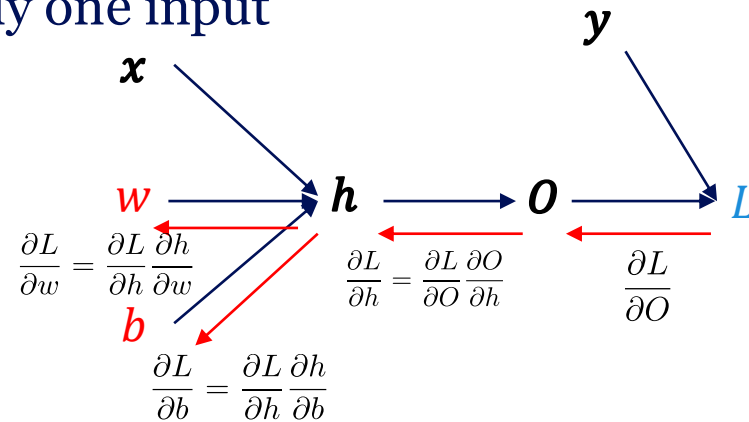
$$\frac{\partial L}{\partial O} = (O - y)$$

$$\frac{\partial O}{\partial h} = g'(h)$$

$$\frac{\partial h}{\partial w} = x, \frac{\partial h}{\partial b} = 1$$

Step 2 - Univariate Chain rule

Consider a single node with only one input



The univariate chain rule:

$$\frac{\partial L}{\partial w} = \boxed{\frac{\partial L}{\partial o} \frac{\partial o}{\partial h}} \frac{\partial h}{\partial w}$$

$$\frac{\partial L}{\partial b} = \boxed{\frac{\partial L}{\partial o} \frac{\partial o}{\partial h}} \frac{\partial h}{\partial b}$$

$$\frac{\partial h}{\partial w} = x, \frac{\partial h}{\partial b} = 1$$

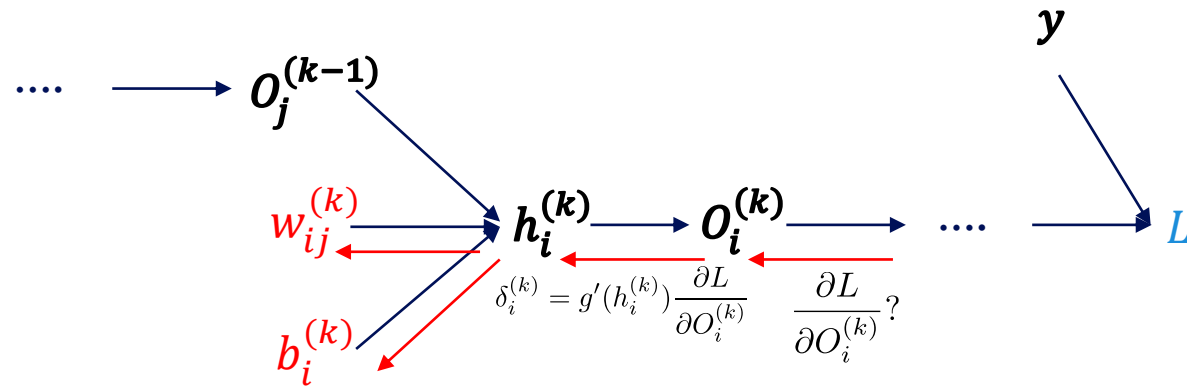
If we define: $\delta = \boxed{\frac{\partial L}{\partial o} \frac{\partial o}{\partial h}} = \frac{\partial L}{\partial o} g'(h)$

$$\frac{\partial L}{\partial w} = \delta x$$

$$\frac{\partial L}{\partial b} = \delta$$

Step 3 – Defining the error term

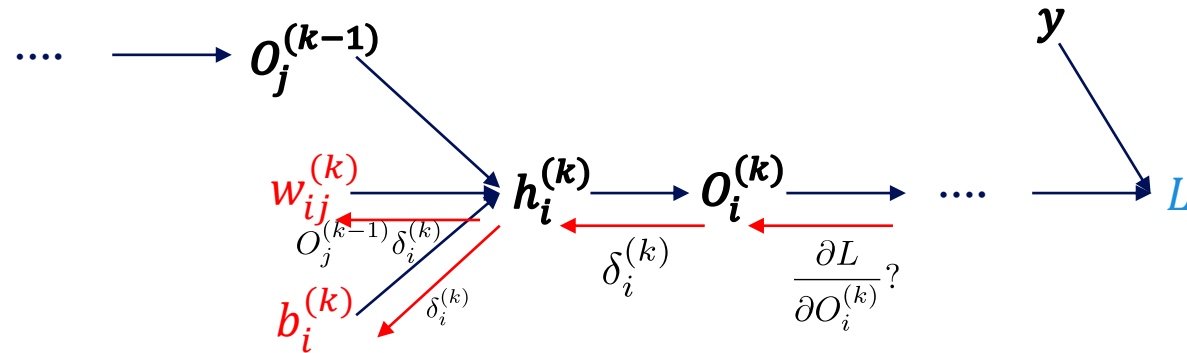
- Let's reformulate this simple example:



- Define an error term (how much the loss would change if we perturb h ?)

$$\delta_i^{(k)} = \frac{\partial L}{\partial h_i^{(k)}} = \frac{\partial L}{\partial o_i^{(k)}} \frac{\partial o_i^{(k)}}{\partial h_i^{(k)}} = g'(h_i^{(k)}) \frac{\partial L}{\partial o_i^{(k)}}$$

Step 3 – Compute gradients for layer k



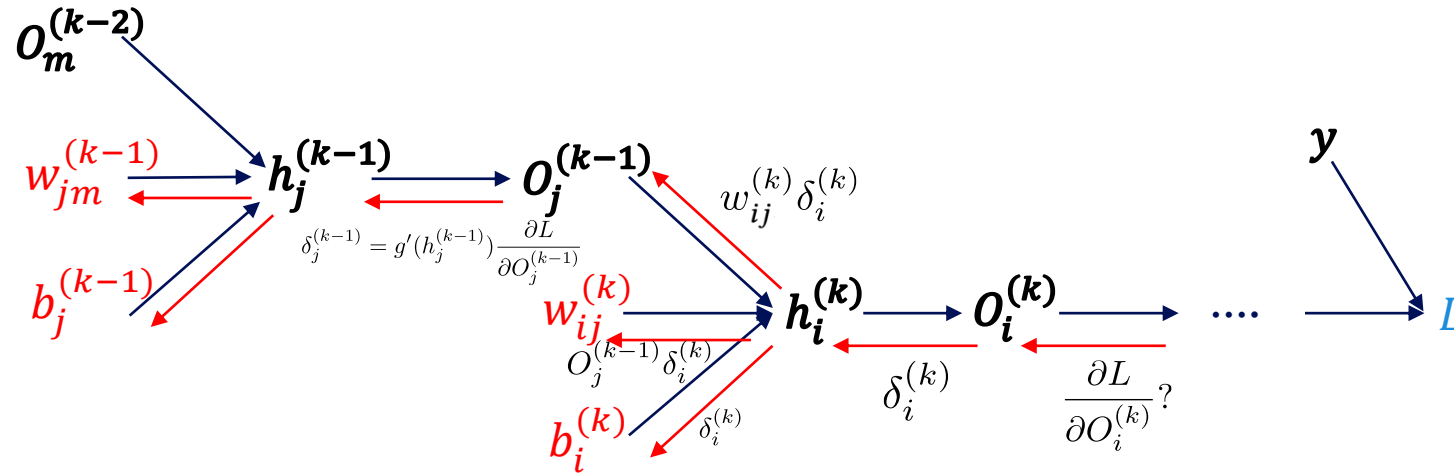
Recall:
$$h_i^{(k)} = \sum_{j=1}^{n_{k-1}} w_{ij}^{(k)} O_j^{(k-1)} + b_i^{(k)}$$

$$\frac{\partial L}{\partial w_{ij}^{(k)}} = \frac{\partial L}{\partial h_i^{(k)}} \frac{\partial h_i^{(k)}}{\partial w_{ij}^{(k)}} = O_j^{(k-1)} \delta_i^{(k)}$$

$$\frac{\partial L}{\partial b_i^{(k)}} = \frac{\partial L}{\partial h_i^{(k)}} \frac{\partial h_i^{(k)}}{\partial b_i^{(k)}} = \delta_i^{(k)}$$

Hebb's rule!

Step 4 – Backpropagation from layer k to $k-1$

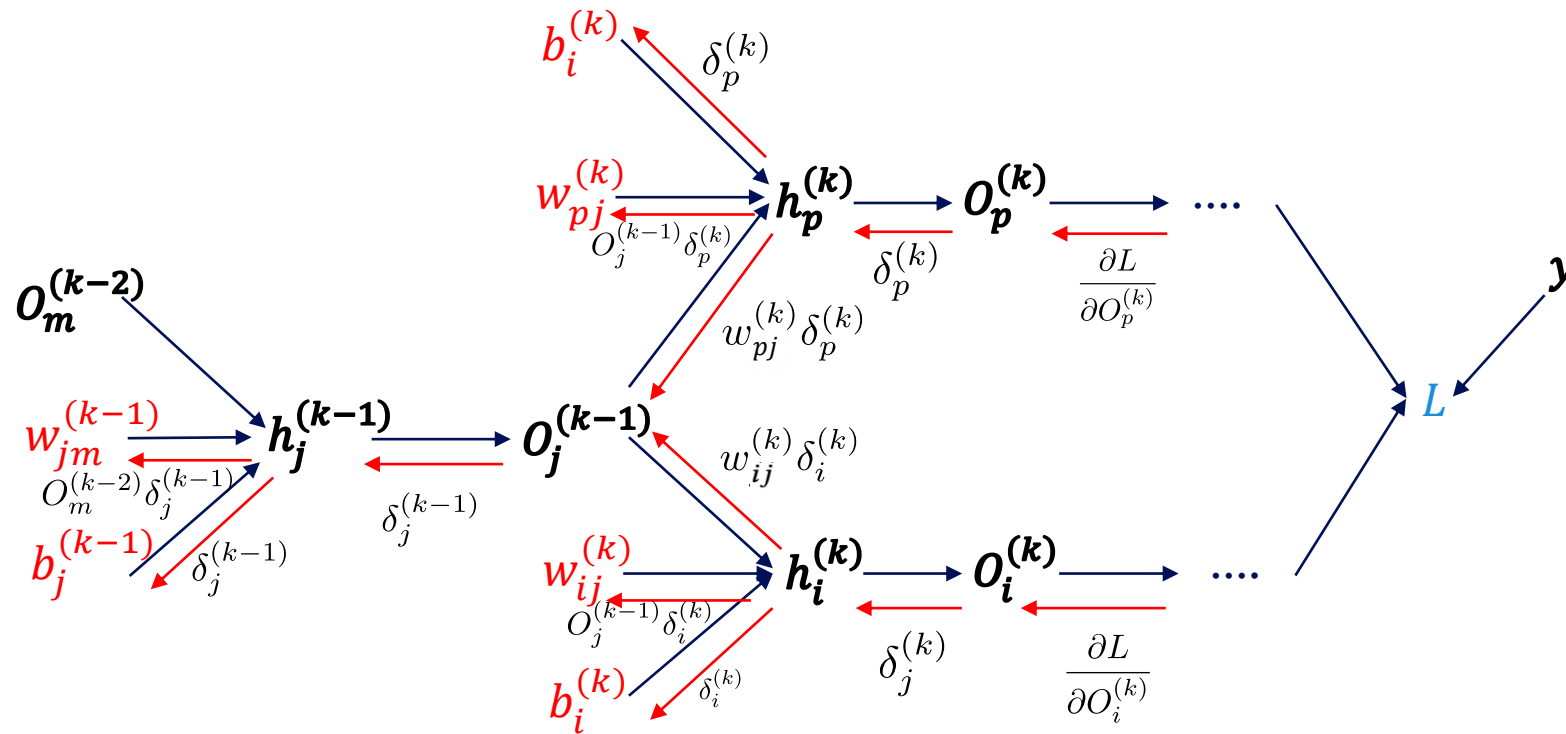


$$\frac{\partial L}{\partial O_j^{(k-1)}} = \frac{\partial L}{\partial h_i^{(k)}} \frac{\partial h_i^{(k)}}{\partial O_j^{(k-1)}} = w_{ij}^{(k)} \delta_i^{(k)}$$

$$\delta_j^{(k-1)} = g'(h_j^{(k-1)}) \frac{\partial L}{\partial O_j^{(k-1)}} = g'(h_j^{(k-1)}) w_{ij}^{(k)} \delta_i^{(k)}$$

Recall:
$$h_i^{(k)} = \sum_{j=1}^{n_{k-1}} w_{ij}^{(k)} O_j^{(k-1)} + b_i^{(k)}$$

Step 5 – Multivariate Chain Rule



$$\frac{\partial L}{\partial O_j^{(k-1)}} = \sum_{i=1}^{n_k} w_{ij}^{(k)} \delta_i^{(k)}$$

The BP Algorithm

1) Forward phase - for each input-output pair $(\vec{x}, y)$, starting from the first layer, calculate and store the quantifies $O_i^{(k)}, h_i^{(k)}$ for each node.

2) Backward phase - for the last/output layer compute the error term: $\delta^{\text{out}} = g'(h^{\text{out}})(y - O)$

While until the first layer:

- 1) Compute the gradient for each weight and bias: $\frac{\partial L}{\partial w_{ij}^{(k)}} = O_j^{(k-1)} \delta_i^{(k)}$
- 2) Backpropagate the error term to each node of the previous layer: $\delta_j^{(k-1)} = g'(h_j^{(k-1)}) \sum_{i=1}^{n_k} w_{ij}^{(k)} \delta_i^{(k)}$
- 3) $k \leftarrow k - 1$

3) Gradient descent – for each weight (incl. bias)

$$w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \eta \frac{\partial L}{\partial w_{ij}^{(k)}}$$

The BP Algorithm

Question:

Do I feed all the data points into
the following three steps?

Gradient Descent

- 1) **Batch Gradient Descent** – take the whole data set, compute the loss and gradient at each point, then calculate the average gradient

$$\text{mean} \left(\frac{\partial L(\vec{x}(1))}{\partial w_{ij}^{(k)}}, \frac{\partial L(\vec{x}(2))}{\partial w_{ij}^{(k)}}, \dots, \frac{\partial L(\vec{x}(N))}{\partial w_{ij}^{(k)}} \right)$$

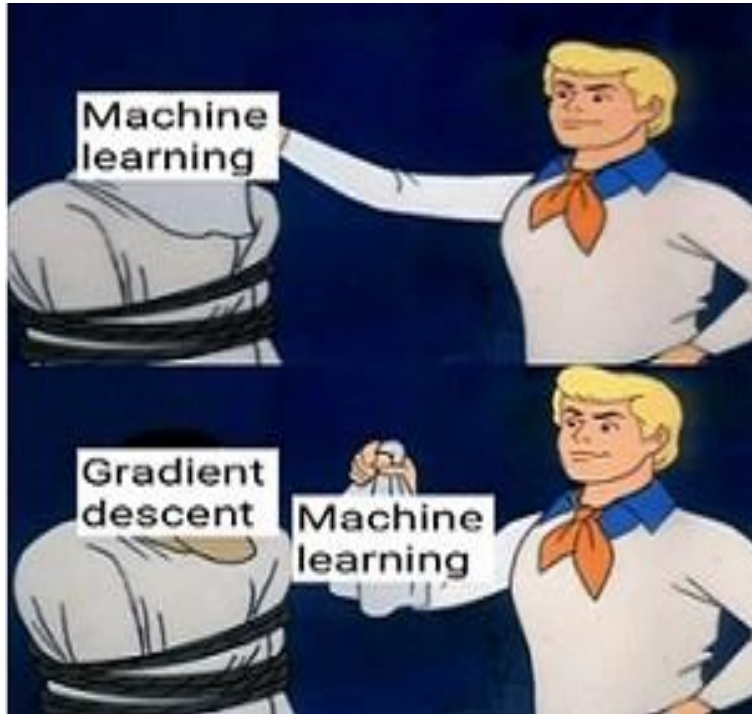
- Pros: precise, less random
- Cons: additional local storage

- 2) **Stochastic Gradient Descent** – take a random data point, compute gradient with it

- Pros: less storage, superior for redundant training data
- Cons: increased oscillations in the convergence

- 3) **Mini-batch Gradient Descent** – take a (random) subset of the whole data to compute the gradient only on this subset

Gradient Descent



Neural networks are trained in a series of **epochs**.

Each epoch consists of one forward pass and one backward pass over **the whole data set**.

In mini-batch GD, the processing of each subset of data points is called a **step**.

Mini-batch Gradient Descent

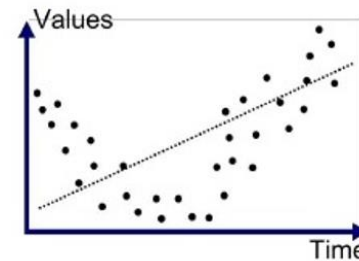
1) Batch size – Total number of training data points present in a single batch for computing the gradient.

- Batch sizes that are multiples of powers of 2 are common, e.g., 32, 64, 128, and so on

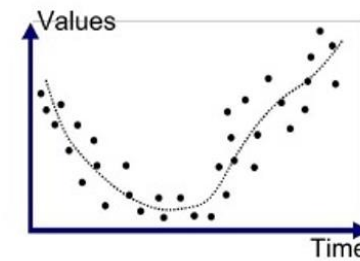
2) Iteration/step – the number of batches needed to complete one epoch.

3) We need more than one epoch, why?

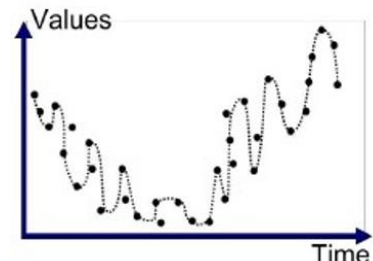
- One epoch leads to underfitting since GD is an iterative process, which might need many iterations to find good weights
- Subsets are taken randomly from the whole training set.



Underfitted



Good Fit/Robust



Overfitted

Mini-batch Gradient Descent

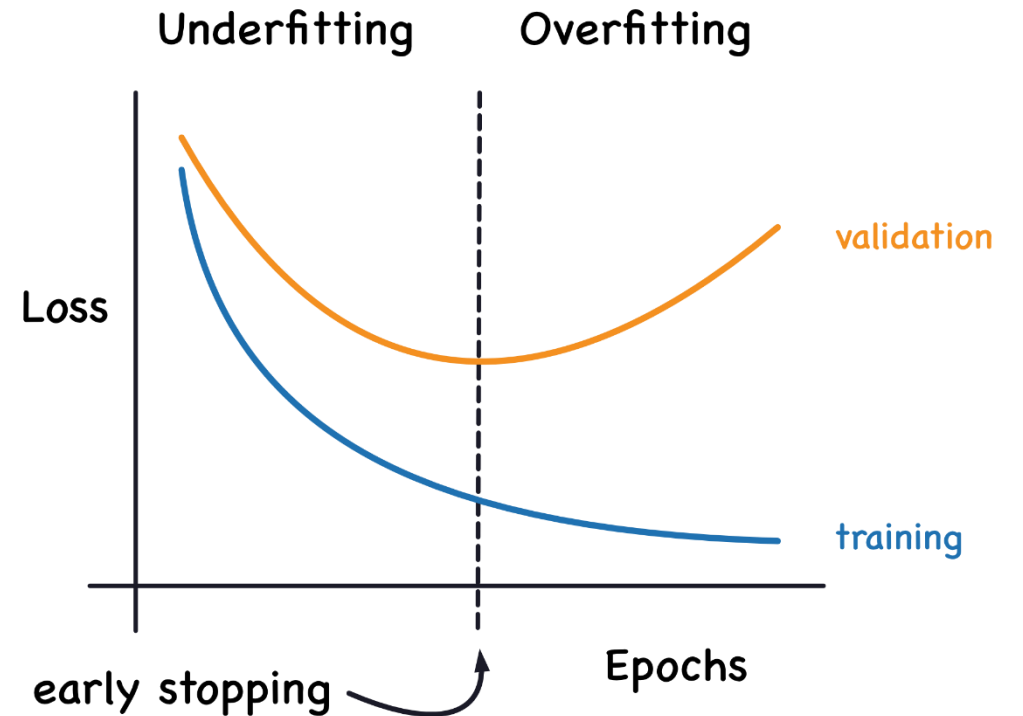
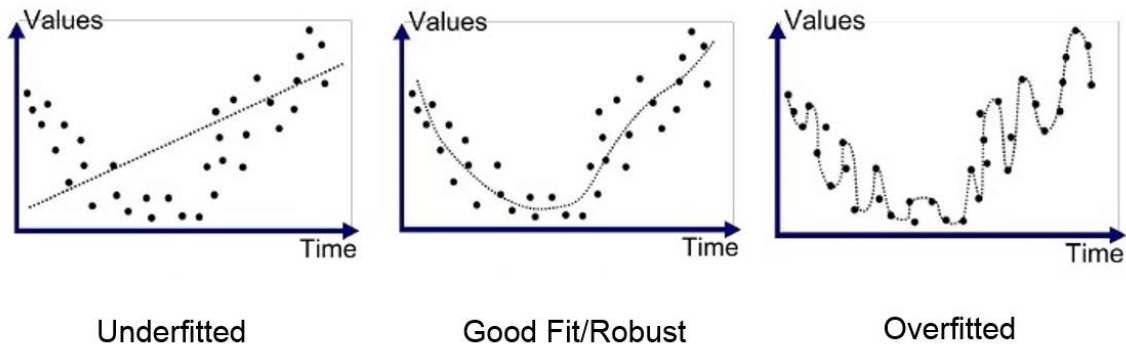
For $i = 1, 2, \dots, M$; do

Repeat until every data point in the training set is processed

- Sample a batch B of data points of size n uniformly at random (uniformly at random) from the training set
- Perform the forward pass on batch B
- Perform the backward pass on batch B (backpropagate the error)
- Gradient descent with batch B

done

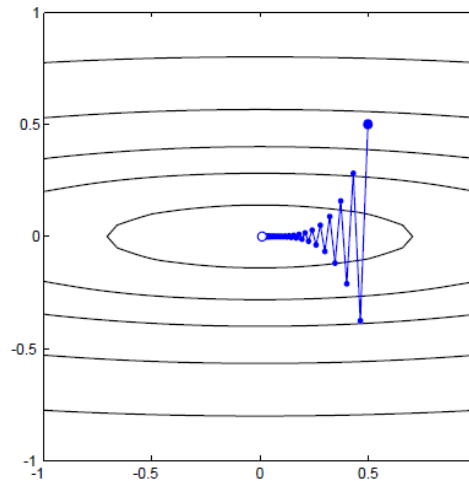
Hmm.. Overfitting?



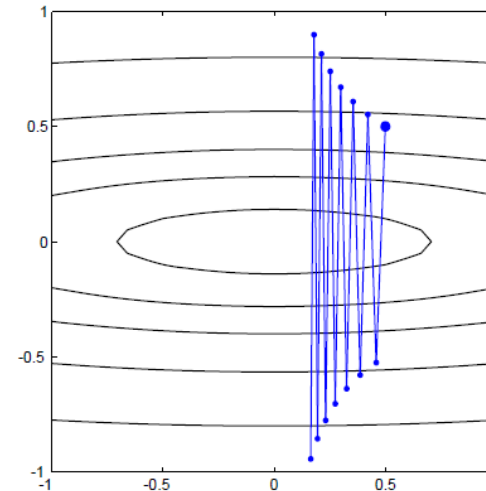
Gradient Descent Algorithms

This is a simple rule... Recall what are the issues with it?

$$w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \eta \frac{\partial L}{\partial w_{ij}^{(k)}}$$



$\eta = 0.035$

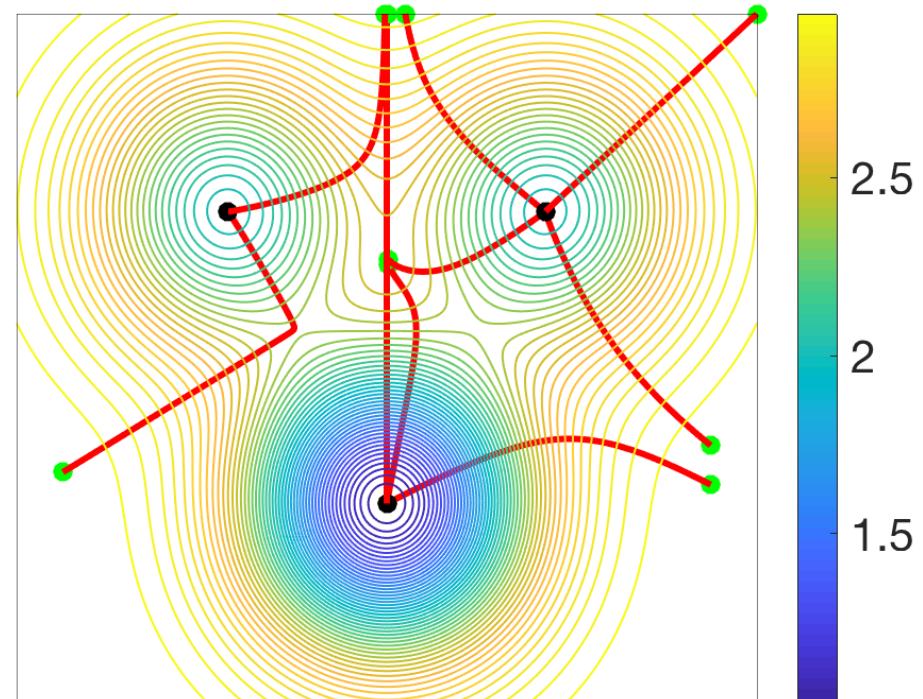
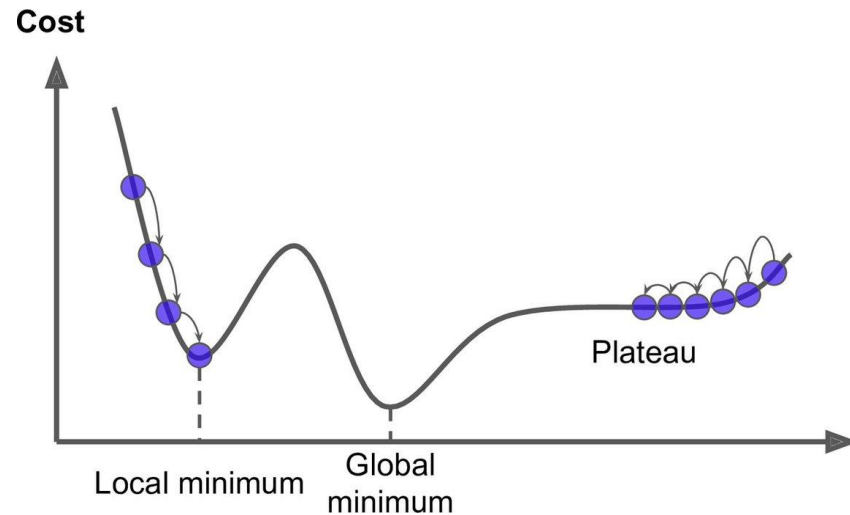


$\eta = 0.041$

Gradient Descent Algorithms

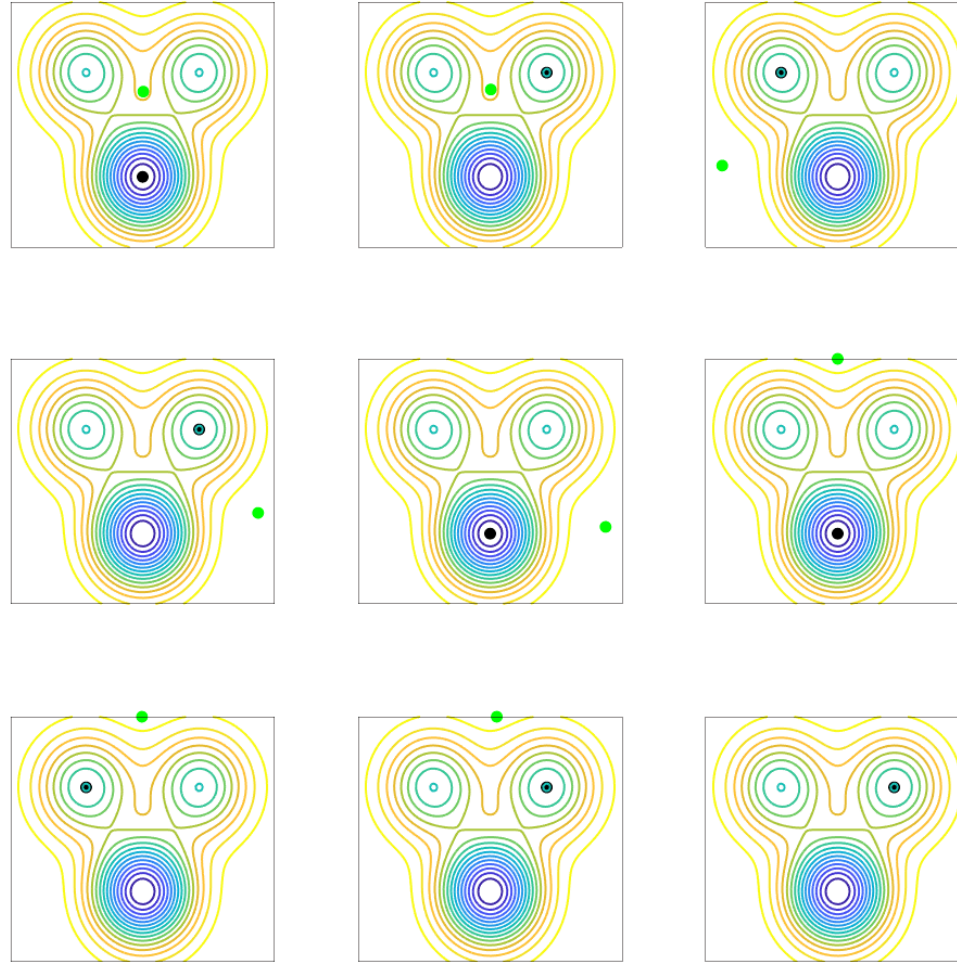
It can also get stuck in local optimum!

$$w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \eta \frac{\partial L}{\partial w_{ij}^{(k)}}$$



Courtesy of <https://francisbach.com/gradient-flows/>

Gradient Descent Algorithms



Courtesy of <https://francisbach.com/gradient-flows/>

Gradient Descent Algorithms

- *Momentum*
- Nesterov accelerated gradient
- Adagrad
- Adadelata
- RMSprop
- Adam
- AdaMax
- Nadam
- AMSGrad
- ...

Gradient Descent with Momentum

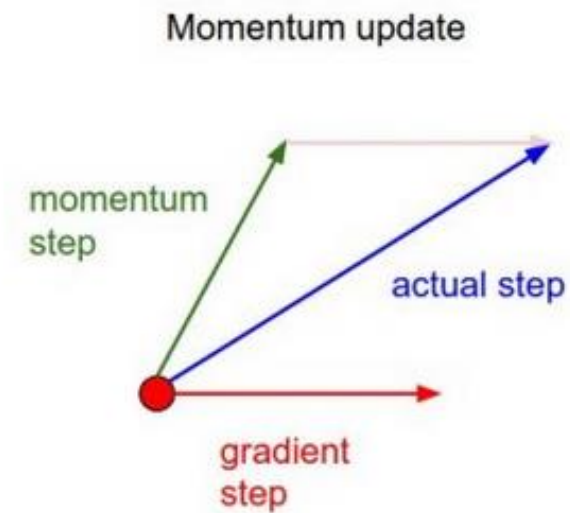
Momentum

$$v_{ij}^{(k)} \leftarrow \gamma v_{ij}^{(k)} + \eta \frac{\partial L}{\partial w_{ij}^{(k)}}$$

$$w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - v_{ij}^{(k)}$$

$v_{ij}^{(k)}$ is set to zero initially

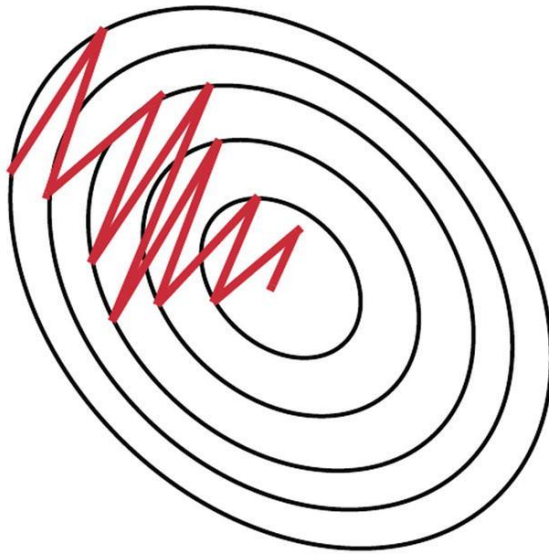
$$\gamma, \eta \in (0, 1)$$



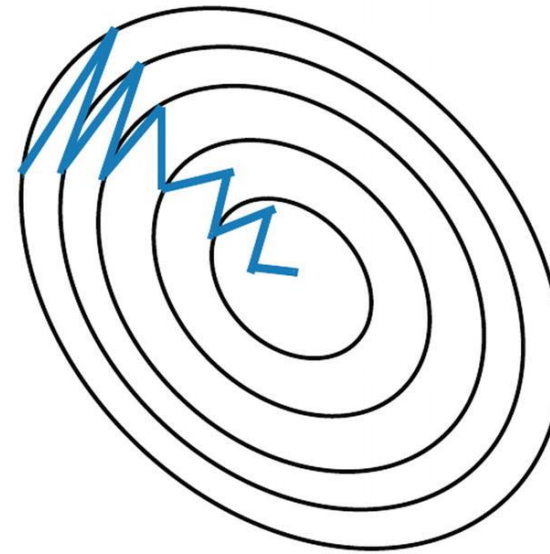
Gradient Descent with Momentum

- Momentum

- Reduce oscillation/instability
- Can also help with escaping from local optimum



Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum



Thank you – Any questions?



Universiteit
Leiden
The Netherlands

e.raponi@liacs.leidenuniv.nl

Discover the world at Leiden University